

MYSTEN LABS · SECURITY REVIEW

Mysten Labs suiUSDe Review.

A security review of suiUSDe Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Mysten Labs	suiUSDe Review	Guardian	December 10 – December 19, 2025	December 29, 2025

§ 00

Contents.

47 pages — overview, confidence ranking, scope, 33 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 33 findings	P. 10
§ 05	Disclaimer	P. 46
§ 06	About Guardian	P. 47

§ 01

Overview.

Guardian reviewed suiUSDe Review across a single review round. The final report includes 4 Low, 29 Informational findings.

AUDIT FIRM	METHODOLOGY
Guardian	Smart contract · Manual · Main Review
CLIENT	
Mysten Labs	
PROJECT	
suiUSDe Review	
PREPARED BY	
Guardian	
AUDIT WINDOW	
December 10 – December 19, 2025	
FINAL REPORT	
December 29, 2025	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	0	0	0	0	0
Low	4	0	2	0	2
Informational	29	0	21	0	8
All severities	33	0	23	0	10

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed suiUSDe Review.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
L-01	Replenish Does Not Validate Collateral Status	Validation	LOW	ACKNOWLEDGED	p. 10
L-02	Malicious Nonce Strings	Warning	LOW	ACKNOWLEDGED	p. 11
L-03	Missing Global Pause Capability	Unexpected Behavior	LOW	RESOLVED	p. 12
L-04	Missing Version Check	Access Control	LOW	RESOLVED	p. 13
I-01	Aggregated Oracle Allows Arbitrage	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 14
I-02	Oracle Staleness Validated Only On Commitment	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 16
I-03	AggregateOracle Multiple Sources Incompatibility	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 17
I-04	Unnecessary assert_is_enabled Check	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 18
I-05	Typo	Typo	INFORMATIONAL	RESOLVED	p. 19
I-06	Treasury Created With Incorrect Version Config	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 20
I-07	Redundant oracle_limits validation	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 21
I-08	Validation Allows Same Maximum And Minimum	Validation	INFORMATIONAL	RESOLVED	p. 22
I-09	fee_is_charged Inaccuracy	Unexpected Behavior	INFORMATIONAL	RESOLVED	p. 23

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-10	Lacking minAmountOut Validation	Validation	INFORMATIONAL	RESOLVED	p. 24
I-11	Lack of version validation	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 25
I-12	Blacklist DoS During Upgrade	DoS	INFORMATIONAL	ACKNOWLEDGED	p. 26
I-13	Oracle Can Be Accidentally Bricked	DoS	INFORMATIONAL	ACKNOWLEDGED	p. 27
I-14	Benefactors Cannot Be Configured Before Live	Configuration	INFORMATIONAL	ACKNOWLEDGED	p. 28
I-15	Idempotent Configurations Emit Events	Events	INFORMATIONAL	ACKNOWLEDGED	p. 29
I-16	Overrestrictive Oracle Buffer Validation	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 30
I-17	Lacking Upper Oracle Timestamp Bound	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 31
I-18	Unnecessary Treasury Parameter Mutability	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 32
I-19	Oracle Reduces Precision	Rounding	INFORMATIONAL	ACKNOWLEDGED	p. 33
I-20	Lacking Overflow Check	Validation	INFORMATIONAL	RESOLVED	p. 34
I-21	Max Peg Price Allows For Overflow	Validation	INFORMATIONAL	RESOLVED	p. 36
I-22	fee_is_charged Can Be Misleading	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 37
I-23	Mutable Oracle Reference May Congest Actions	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 38
I-24	Pyth Arbitrage Window	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 39
I-25	Epoch And Period Counter Reset Permanently	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 40
I-26	Misleading Documentation	Documentation	INFORMATIONAL	RESOLVED	p. 42
I-27	Mismatching MAX_ORACLE_AGE	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 43
I-28	Lacking Sanity Overflow Check	Validation	INFORMATIONAL	RESOLVED	p. 44

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-29	Custody Transfer Ignores Collateral Enabled Flag	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 45

LOW **L-01**

ACKNOWLEDGED

Replenish Does Not Validate Collateral Status

CATEGORY Validation
LOCATION permissionless.move
REVIEW Main Review

DESCRIPTION

The `replenish` function does not validate the active status of a collateral that is being replenished. Because of this it might be possible for any caller to accidentally replenish a disabled collateral.

RECOMMENDATION

Consider validating that the collateral is enabled in the `replenish` function.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Malicious Nonce Strings

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Nonces in the Move implementation are arbitrary strings with no distinct limit on their length.

This means that they could be used to inject malicious code into whatever systems are consuming the mint/redeem events that emit this nonce value.

— RECOMMENDATION

Consider adding a reasonable length validation on the nonce string to decrease the possibility of them being able to be used as malicious injection.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

RESOLVED

Missing Global Pause Capability

CATEGORY Unexpected Behavior
LOCATION sui_usde.move
REVIEW Main Review

DESCRIPTION

When creating the suiUSDe token in the `init_treasury` function the `make_regulated` function receives an `allow_global_pause` value of `true` which indicates that the protocol would like to have the ability to pause usage of the suiUSDe coin across the entire network with the `deny_list_v2_enable_global_pause` function.

However in the `denylist_manager` module there are only functions exposed to add addresses and remove addresses from the denylist. None that expose the ability to pause and unpause the global pause functionality which has been whitelisted for the suiUSDe token.

RECOMMENDATION

Consider exposing the global pause and unpause functionality through the `deny_list_v2_enable_global_pause` function if it is desired to have this capability.

RESOLUTION

Status: Resolved.

LOW **L-04**

RESOLVED

Missing Version Check

CATEGORY Access Control
LOCATION permissionless.move
REVIEW Main Review

DESCRIPTION

The `new_oracle_proof` and `commit_pyth_price` functions do not implement a config version check from the Treasury, and thus are callable even after a package upgrade has occurred and the version has been bumped.

If important updates were made to the Oracle in the upgrade, as long as no new logic has been added in the upgrade that prevents the old Oracle logic from being used, then the old oracle logic can be used to create unexpected OracleProof objects which could contain prices that do not meet the standards of the new version of the contracts.

RECOMMENDATION

Enforce the config version check in the `new_oracle_proof` and `commit_pyth_price` functions.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-01

ACKNOWLEDGED

Aggregated Oracle Allows Arbitrage

CATEGORY	Validation
LOCATION	aggregated_oracle.move
REVIEW	Main Review

DESCRIPTION

In the `commit_pyth_oracle_price` flow it is possible for a benefactor user to control the end outcome of the median result and potentially arbitrage the system when multiple oracle feeds are supported because of a few factors.

The `commit_pyth_oracle_price` function will commit "invalid" price types to the proof to indicate that the price was provided for the relevant feed but was found to be invalid. The `validate_and_get_valid_prices` function which retrieves the valid prices on usage tosses out any entries that were deemed to be invalid upon commitment.

The user can control whether a price is considered valid on commitment by choosing to pass a stale Pyth update versus passing an up to date one.

This means a benefactor could provide a commitment such as the following:

- Assume `minimum_sources` is assigned as 2 and there are 3 total feeds configured
- For feed 0 (A pyth feed) the value is \$1.01, the user provides a price that is 5 minutes old, it is considered invalid
- For feed 1 the value is \$1.00, and the price is valid
- For feed 2 the value is \$0.99, and the price is valid
- The computed median is the average of feed 1 and feed 2 which is \$0.995

The benefactor could mint using this price, and then redeem using a different price they commit:

- For feed 0 (A pyth feed) the value is \$1.01, the user now provides a price that is only 10 seconds old, it is considered valid
- For feed 1 the value is \$1.00, and the price is valid
- For feed 2 the value is \$0.99, and the price is valid
- The computed median is now feed 1, which is \$1.00
- If the spread between these two median results is large enough, the user could gain an immediate arbitrage on the system

I-01 Aggregated Oracle Allows Arbitrage

CONTINUED

— RECOMMENDATION

When multiple feeds are supported by the aggregate oracle, consider requiring that all pull based feeds where the user provides their own price update have valid staleness.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Oracle Staleness Validated Only On Commitment

CATEGORY	Validation
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

At the time of price usage, in the `latest_price` function, the earliest price from all oracle sources is compared against the `collateral_limits` maximum age configuration.

However this is a different configuration than the `oracle_limits.max_age_ms`. Currently these are to be configured to the same value, but in the event that they are not configured to the same value this may allow prices to be committed in a fresh state and then become stale by the aggregate oracle's standards by the time the proof is used.

More notably, when multiple oracle feeds are supported the `collateral_limits` max staleness will not be able to apply to each feed individually and so therefore must take the maximum staleness value of all of the feeds. Therefore if there is an onchain aggregator in the list of supported feeds, which has a heartbeat of 12 hours, then the `collateral_limits` max staleness must be configured to at least 12 hours.

This is far too long for a pyth push based feed, and would allow a user to knowingly or unknowingly commit a pyth push price and wait up to 12 hours before using it, at which point it can be notably stale and out of line with the current market values.

RECOMMENDATION

In the future when multiple feeds are supported by the Aggregate Oracle, also perform per-feed level validations on staleness at usage time in the `latest_price` function.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

AggregateOracle Multiple Sources Incompatibility

CATEGORY	Validation
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The Aggregated Oracle is intended to function with multiple feed sources and compute a median result of those provided. However the validation on staleness is only applied at the aggregate oracle level.

When multiple feeds are supported they may have different staleness tolerance requirements and thus should have a feed level staleness validation.

— RECOMMENDATION

While the existing setup only uses a single pyth feed, in the future when an upgrade is made to support multiple feeds, an individual feed level staleness validation should be introduced as a broader aggregate feed validation may not be able to be strict enough for some feeds in the aggregate oracle.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Unnecessary assert_is_enabled Check

CATEGORY Superfluous Code
LOCATION benefactor.move
REVIEW Main Review

DESCRIPTION

In the `validate_states` function the `benefactor_config.assert_is_enabled` method is invoked to validate that the benefactor is enabled, however the following `benefactor_config.add_nonce` call already includes such validation in its function body. Therefore the previous bespoke invocation is unnecessary.

RECOMMENDATION

Consider removing the `benefactor_config.assert_is_enabled` inside the `validate_states` function as this is already performed within the `add_nonce` invocation.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-05**

RESOLVED

Typo

CATEGORY	Typo
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Throughout the codebase Committed is misspelled as `Commited`.

RECOMMENDATION

Correct these instances to Committed.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-06**

ACKNOWLEDGED

Treasury Created With Incorrect Version Config

CATEGORY Validation
LOCATION treasury.move
REVIEW Main Review

— DESCRIPTION

In the treasury.move file the create function for the Treasury object does not validate that the config provided uses an up to date version.

As a result, a treasury can be errantly created that is immediately invalid and outdated.

— RECOMMENDATION

Consider adding validation that the config provided to the create function is using an up to date config version.

PROOF OF CONCEPT <https://github.com/GuardianOrg/suiusde-team1-1765190388850/pull/1>

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

ACKNOWLEDGED

Redundant oracle_limits validation

CATEGORY Superfluous Code
LOCATION aggregated_oracle.move
REVIEW Main Review

DESCRIPTION

The `validate()` call in `set_minimum_sources` revalidates `oracle_limits`, even though limits aren't changed there. Limits are already validated at creation and when setters are used, so this check is redundant in this path.

RECOMMENDATION

Remove this oracle limits validation in `set_minimum_sources` path

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-08**

RESOLVED

Validation Allows Same Maximum And Minimum

CATEGORY	Validation
LOCATION	oracle_limits.move
REVIEW	Main Review

— DESCRIPTION

In the oracle_limits module within the suiusde package, the validate function which is used to check the limit configurations allows the max_price to be equal to the min_price.

This however is unlikely to be a realistic valid configuration and does not align with the validations made on the Solidity implementation.

— RECOMMENDATION

Consider enforcing more strict validations that require the max_price to be strictly greater than the min_price.

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-09

RESOLVED

fee_is_charged Inaccuracy

CATEGORY	Unexpected Behavior
LOCATION	benefactor.move
REVIEW	Main Review

DESCRIPTION

In the mint and redeem functions for the benefactor, the `fee_is_charged` value is intended to indicate whether the collateralization of the action was better than a direct oracle exchange for the protocol, e.g. that the fee was charged.

However the `fee_is_charged` value is also assigned to true when the `one_to_one_amount_out` is equivalent to the `oracle_amount_out`. When this is the case no fee was not charged relative to the oracle price, so the `fee_is_charged` value should instead be false to indicate this.

RECOMMENDATION

Consider updating the `fee_is_charged` definition to be a strict less than comparison, so that it is only true when some nonzero fee is charged between the `one_to_one_amount_out` and `oracle_amount_out`.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-10**

RESOLVED

Lacking minAmountOut Validation

CATEGORY	Validation
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

In the Solidity implementation of the stablecoin system, there is validation which prevents the user from providing a minAmountOut of 0 accidentally. However the Move implementation does not implement such a validation.

RECOMMENDATION

Consider if the minimum amount out should be required to be nonzero to protect against potential misconfigurations.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-11**

ACKNOWLEDGED

Lack of version validation

CATEGORY	Validation
LOCATION	admin.move
REVIEW	Main Review

— DESCRIPTION

The `set_version` function in the admin module allows setting any values as version, including older or current versions. This can lead to unintended downgrades or redundant updates, potentially causing compatibility or versioning issues.

— RECOMMENDATION

In `set_version` function only allow strictly increasing values to version to prevent accidental or malicious downgrades and redundant updates

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-12**

ACKNOWLEDGED

Blacklist DoS During Upgrade

CATEGORY	DoS
LOCATION	denylist_manager.move
REVIEW	Main Review

DESCRIPTION

In the `add_to_denylist` function the treasury's `deny_cap_mut` function and other similar functions which validate the config version against the current breaking version is used.

This means that when in the middle of an upgrade process, or generally when the contracts for the minting system have been temporarily disabled, the denylist is not able to be updated.

This may create a risk of a lack of ability to respond to an incident if it occurs during this temporary period, or may create operational issues with the ability to adhere to compliance of KYC etc... during this period.

Furthermore, the `add_to_denylist` requires a `&Auth` reference value which can only be obtained by the `new_auth` function which also performs the `assert_valid_version` validation.

RECOMMENDATION

Consider implementing and using versions of the `has_benefactor`, `benefactor_mut`, and `deny_cap_mut` functions that do not validate the config version in order to support blacklisting addresses in emergency scenarios even when the minting system is temporarily disabled with the `add_to_denylist` function.

Furthermore, consider using the following authentication check in the `add_to_denylist` function instead of a typed check:

```
treasury.roles_for_version_update().assert_is_authorized<DenylistManagerRole>(ctx.sender());
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/suiusde-team1-1765190388850/pull/2>

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-13**

ACKNOWLEDGED

Oracle Can Be Accidentally Bricked

CATEGORY	DoS
LOCATION	aggregated_oracle.move
REVIEW	Main Review

— DESCRIPTION

In the EVM implementation of the stablecoin system the oracle admin cannot accidentally brick the oracle by assigning the minimum oracle sources to be above the available number of oracles in the aggregate oracle.

However in the Move implementation there is no such validation that prevents the oracle manager from assigning a `minimum_sources` value that is above the number of supported feeds.

Furthermore, in the `remove_feed` function there is no validation that requires that the number of remaining oracle feeds exceeds the minimum. This also deviates from the EVM implementation.

— RECOMMENDATION

Consider if these validations should be implemented to be in line with the EVM implementation.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-14**

ACKNOWLEDGED

Benefactors Cannot Be Configured Before Live

CATEGORY	Configuration
LOCATION	benefactor_manager.move
REVIEW	Main Review

— DESCRIPTION

In the EVM implementation of the minting system Benefactor configurations such as mint and redeem fees, benefactor limits, and fee exemption statuses can be configured by the benefactor manager before a benefactor address has been whitelisted as a benefactor.

This is however not the case in the Move implementation as the `benefactor_mut` function requires that the address is contained in the `treasury.benefactors` Bag.

— RECOMMENDATION

This is a fine behavior, however deviates from the EVM implementation. For any integrators who are supporting and working with the EVM implementation this may be unexpected.

Consider documenting this behavior, or at least be aware of this discrepancy compared to the EVM version.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-15**

ACKNOWLEDGED

Idempotent Configurations Emit Events

CATEGORY	Events
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

In the EVM implementation the benefactor manager configuration functions are careful to only emit events that indicate a successful configuration when the newly configured value is different from the one that existed before it.

The Move implementation however does not do this and always emits the relevant event even if the configuration did not change anything.

— RECOMMENDATION

Be aware of this discrepancy between the EVM and Move implementations.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-16**

ACKNOWLEDGED

Overrestrictive Oracle Buffer Validation

CATEGORY Validation
LOCATION oracle_helpers.move
REVIEW Main Review

— DESCRIPTION

In the `calculate_usd_price_internal` function the oracle buffer validation requires that:
`constants::oracle_buffer() + target_decimals > oracle_decimals` with strict inequality.

However the equality case is a valid scenario as well which does not cause an underflow when calculating the `exponent_with_buffer` result, though perhaps unlikely in real world configurations.

— RECOMMENDATION

Consider using a greater than or equal to comparison for the validation to allow the full range of technically valid configurations.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-17**

ACKNOWLEDGED

Lacking Upper Oracle Timestamp Bound

CATEGORY	Validation
LOCATION	aggregated_oracle.move
REVIEW	Main Review

— DESCRIPTION

Since Pyth prices originate from offchain it is entirely possible that the oracle timestamp is in the future relative to the trusted Sui clock.

The Solidity version accounts for this by validating that prices are no more than 15 seconds into the future, as anything further would be unexpected and likely point to some mis-configuration.

— RECOMMENDATION

Consider adding a future timestamp threshold, whereby if a Pyth price result is from too far in the future than would be normally realistic for time mismatch between Pyth's offchain system and Sui's trusted clock the the transaction is aborted.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-18**

ACKNOWLEDGED

Unnecessary Treasury Parameter Mutability

CATEGORY	Best Practices
LOCATION	auth.move
REVIEW	Main Review

— DESCRIPTION

In the `new_auth` function the treasury parameter is passed as a mutable reference, however it is not used in a way that requires mutability.

— RECOMMENDATION

Mark the treasury variable as an immutable parameter.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-19**

ACKNOWLEDGED

Oracle Reduces Precision

CATEGORY	Rounding
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

In the EVM implementation the oracle feed contracts used in the aggregate oracle contract scale the price result up to 18 decimals for an internal price representation that has 18 decimals of precision.

However the oracle implementation in the Move implementation scales price results (often times down) to 6 decimals. For example, the pyth feed for USDC has 8 decimals of price precision, which would be scaled down to 6, thus destroying some precision.

— RECOMMENDATION

Consider using a higher magnitude scale such as 8 decimals instead of down to 6 for the internal price representation to preserve as much price granularity as possible.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-20

RESOLVED

Lacking Overflow Check

CATEGORY	Validation
LOCATION	oracle_helpers.move
REVIEW	Main Review

DESCRIPTION

In the `collateral_to_suiusde_amount` function there is no validation that prevents the final `suiusde_amount` result from overflowing the u64 cast that is made.

This scenario is certainly invalid as the Coin object has a value field of u64 and thus the `totalSupply` cannot exceed this.

However it may technically be possible to reach this state if a certain collateral with lower decimals is used in tandem with the peg price being set extremely low.

Consider the following case:

- `suiusde_decimals` = 6
- `collateral_usd_price` = $1e6$
- `collateral_amount` = 20_000_000e6
- `collateral_decimals` = 6
- `suiusde_peg_price` = 1
- `numerator` = $20_000_000e6 * 1e6 * 10^{**6} = 20e24$
- `denominator` = $1e6 * 1 = 1e6$
- `suiusde_amount` = $20e24 / 1e6 = 20e18 > \text{type}(u64) \text{ maximum}$

Assigning the `suiusde_peg_price` to 1 is unlikely to ever happen and would be considered an mis-configuration in almost all scenarios. However it does produce this overflow case. Furthermore, unusual tokens with less than 6 decimals would increase the possible peg price assignments which can lead to an overflow.

RECOMMENDATION

Consider validating that the numerator / denominator result is not larger than the maximum u64 as a safety check in the `collateral_to_suiusde_amount` function. Furthermore, consider validating that the peg price is above some reasonable minimum which would also prevent this possibility.

I-20 Lacking Overflow Check

CONTINUED

— **RESOLUTION**

Status: Resolved.

INFORMATIONAL I-21

RESOLVED

Max Peg Price Allows For Overflow

CATEGORY	Validation
LOCATION	oracle_helpers.move
REVIEW	Main Review

DESCRIPTION

The maximum peg price validation allows a peg price of up to 1,000 with six decimal places. However under certain circumstances, for collateral tokens that have higher decimals such as 9 decimals, matching that of the Sui token, an overflow case can still occur in the `suiusde_to_collateral_amount` function.

Consider the following scenario:

- `suiusde_peg_price` = 1_000e6
- `suiusde_amount` = 20_000_000e6
- `collateral_decimals` = 1e9
- `collateral_usd_price` = 1e6
- `suiusde_decimals` = 1e6
- `numerator` = 1_000e6 * 20_000_000e6 * 1e9 = 20e30
- `denominator` = 1e6 * 1e6 = 1e12
- `collateral_amount` = 20e30 / 1e12 = 20e18 > type(u64) maximum

This scenario is unlikely to ever occur, but still shows an overflow with the maximum allowable peg price in place.

RECOMMENDATION

Consider lowering the maximum allowable peg price to a more restrictive value. And furthermore consider adding an overflow check in the `suiusde_to_collateral_amount` just as a safety precaution to prevent any unexpected scenarios.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-22

ACKNOWLEDGED

fee_is_charged Can Be Misleading

CATEGORY	Warning
LOCATION	benefactor.move
REVIEW	Main Review

DESCRIPTION

In the mint and redeem functions the `fee_is_charged` value is a boolean which is set to true whenever the `one_to_one_amount`, which includes a fee, is less than the `oracle_amount_out`.

The `fee_is_charged` value is included in the `emit_order_executed` event emission directly after the `fee_bps` value, which may lead consumers to believe that the entire bps of the order amount were taken as a fee.

However, in the event that `fee_is_charged` is true it just means that some fee was charged relative to the oracle priced valuation, not that the entire bps was charged relative to the oracle valuation.

For example:

- Collateral of order = 10 USDC
- Fee = 3%
- Oracle valuation of order collateral = \$9.90
- One to one valuation of order collateral with fee = \$9.70
- The technical fee levied was only 200 BPS relative to the oracle valuation

RECOMMENDATION

Be aware of this inconsistency and consider emitting the actual delta from the oracle valuation as the fee in the event. Otherwise document this for consumers.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-23**

ACKNOWLEDGED

Mutable Oracle Reference May Congest Actions

CATEGORY	Best Practices
LOCATION	aggregated_oracle.move
REVIEW	Main Review

— DESCRIPTION

In the `new_proof` function the oracle parameter is passed by mutable reference, however the contents of the function do not use the value mutably and it can instead be an immutable reference parameter.

Since the parameter is currently marked as mutable, it must be ordered with other mint and redeem actions which necessarily also require a mutable reference to the oracle through the proof creation process as well as any malicious transactions which simply touch the permissionless `new_oracle_proof` function. In some cases through high usage or malicious flow, this may cause unnecessary congestion for minting and redeeming actions.

— RECOMMENDATION

Make the oracle parameter an immutable reference instead of a mutable one.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-24**

ACKNOWLEDGED

Pyth Arbitrage Window

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Given that the Pyth feed is a push based one where the exact price that is used is controlled by the publisher, within the staleness threshold enforced, the protocol should carefully consider the staleness threshold with respect to the volatility of the collateral being configured.

For example, if a staleness period of 300 seconds or 5 minutes is used, then some assets may realistically move 1% in that timeframe on a semi-regular instance. If the price of a collateral token at $t = 0$ was \$1.01 while it is now at $t = 300$ at \$1.00 this may allow for an arbitrage opportunity depending on the fee configurations of the protocol.

For example:

- Benefactor A uses the price update from $t = 0$ to mint with their collateral token being valued at \$1.01, it is within the staleness tolerance of 300 seconds
- Benefactor A then uses the latest price from the current $t = 300$ to redeem with the same collateral token they are redeeming now being valued at only \$1.00
- As long as the fees do not overcome the 1% delta then the Benefactor A stands to make a risk free profit

— RECOMMENDATION

Consider the staleness threshold for each asset carefully in accordance with the collateral volatility and the corresponding fees being used, also keeping in mind that some benefactors may have little fees with a custom fee or no fees with an exemption.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-25

ACKNOWLEDGED

Epoch And Period Counter Reset Permanently

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

There is a subtle difference between the EVM implementation and the Move implementation with regards to the changing of duration of epochs and periods and the maintaining of the previous duration's counter values.

In the EVM implementation the epoch and period states are not overwritten, as they are keyed based on the duration length. Instead when a new duration is assigned the epoch/period state is pointing to an entirely new storage slot while the old duration's storage slot remains intact.

In the Move implementation there is only one object that stores the current state of the counter for the epoch or period. This means that when the duration is changed the counter values for the epoch and period are overwritten when the relevant mint/redeem action touches this limit.

This subtle difference only manifests itself when a period/epoch duration is changed from A to B and back to A again while still within the same sequence number. In the EVM implementation the previous counter would still hold the original values, in the Move implementation they have been reset to zero.

This behavior also presents itself where some epoch/period values may have been reset to zero while others have not been. This is because counters are only rolled over/reset when they are interacted with. So the global counter is likely to be rolled over in this case, but it may be the case that a benefactor counter or particular collateral counter are not reset since a mint/redeem action may not have happened with them while the package was using the updated B duration.

— RECOMMENDATION

Such a configuration scenario is unlikely to arise, simply be aware of this difference between the Move and EVM implementations.

I-25 Epoch And Period Counter Reset Permanently

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

INFORMATIONAL I-26

RESOLVED

Misleading Documentation

CATEGORY	Documentation
LOCATION	benefactor.move
REVIEW	Main Review

DESCRIPTION

The comment above the `validate_states` function suggests that the order timestamp validation is performed as a part of the `validate_states` execution. However this validation is not performed until the end of the mint/redeem action when `order.finalize()` is invoked.

RECOMMENDATION

Correct the comment above the `validate_states` function to indicate that it does not perform the expiry validation on the order.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-27**

ACKNOWLEDGED

Mismatching MAX_ORACLE_AGE

CATEGORY	Warning
LOCATION	constants.move
REVIEW	Main Review

— DESCRIPTION

In the EVM implementation the `MAX_ORACLE_AGE` is defined as 1441 minutes, or one day plus one minute. In the Move implementation however this is just 1 day represented in milliseconds.

— RECOMMENDATION

Consider if there should be parity between the Move and EVM implementations on the `MAX_ORACLE_AGE` value.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-28

RESOLVED

Lacking Sanity Overflow Check

CATEGORY Validation
LOCATION oracle_helpers.move
REVIEW Main Review

DESCRIPTION

In the `calculate_usd_price_internal` function there is no sanity validation that ensures that the resulting `target_usd_price` fits within a u64 value before being cast as such.

It is incredibly unlikely that the value would exceed u64, however in the event of a misconfiguration or oracle malfunction it would be best to include a validation that aborts if the result is unexpectedly found to be larger than the u64 type.

RECOMMENDATION

Consider adding a sanity validation that prevents logic from continuing if the `target_usd_price` result would overflow the u64 cast.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-29

ACKNOWLEDGED

Custody Transfer Ignores Collateral Enabled Flag

CATEGORY Informational
LOCATION collateral_manager.move
REVIEW Main Review

DESCRIPTION

In collateral_manager, `transfer_to_custody` transfers the redeem balance to the configured `custodian_address`. However, it does not check whether the collateral is enabled, whereas the Solana equivalent withdraw flow gates withdrawals by checking whether the vault is currently enabled.

RECOMMENDATION

Document this as intended behavior, or enforce `assert_is_enabled` in `transfer_to_custody`.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.